

Ethan Krul

ISIN-325

4/30/2026

Competition

1) Display how many purchases and the total amount purchased by each customer using one SQL statement. [1]

SQLQuery6.s...critt (59))* ✕

```
1 -- 1) Display how many purchases and the total amount purchased by each customer using one SQL statement
2
3 -- Count number of purchases and total amount spent per customer
4 SELECT
5     C.CustomerID,
6     C.FirstName,
7     C.LastName,
8     COUNT(P.PurchaseID) AS TotalPurchases, -- counts purchases per customer
9     SUM(P.Amount) AS TotalAmount -- sums total spent
10 FROM Customers C
11 LEFT JOIN Purchases P ON C.CustomerID = P.CustomerID -- include customers with no purchases
12 GROUP BY C.CustomerID, C.FirstName, C.LastName;
```

100 % No issues found

Results Messages

	CustomerID	FirstName	LastName	TotalPurchases	TotalAmount
1	1	Judy	Denth	7	4240.00
2	2	Idris	Elba	6	88.00
3	3	Tom	Hardy	4	2760.00
4	4	Emma	Stone	3	816.00
5	5	Judy	Denth	3	345.00
6	6	Idris	Elba	3	837.00
7	7	Tom	Hardy	3	1355.00
8	8	Emma	Stone	3	140.00
9	9	Chris	Evans	3	9.00
10	10	Scarlett	Johansson	2	400.00
11	11	Robert	Downey	2	1500.00
12	12	Chris	Hemsworth	2	112.00
13	13	Mark	Ruffalo	2	30.00
14	14	Jeremy	Renner	2	70.00
15	15	Henry	Cavill	2	1000.00

2) Display how many customers are in each city using one SQL statement. [1]

SQLQuery6.s...critt (59))* ✕

```
1 -- 2) Display how many customers are in each city using one SQL statement.
2
3 SELECT City, COUNT(*) AS TotalCustomers -- gets total customers in each city
4 FROM Customers
5 GROUP BY City;
```

100 % No issues found

Results Messages

	City	TotalCustomers
1	Boston	5
2	Chicago	6
3	Dallas	2
4	Los Angeles	2
5	Miami	3
6	New York	4
7	Seattle	2

3) Display all the sales each product is tied to, with the output sorted by product and category.
[1]

SQLQuery6.s...critt (59))* ✕

```
1  -- 3) Display all the sales each product is tied to, with the output sorted by product and category.
2
3  -- Show all sales tied to each product, including category
4  SELECT
5      P.ProductName,
6      C.CategoryName,
7      PU.Amount,
8      PU.PurchaseDate
9  FROM Purchases PU
10 JOIN Products P ON PU.ProductID = P.ProductID -- link purchases to products
11 JOIN Categories C ON P.CategoryID = C.CategoryID -- link products to categories
12 ORDER BY P.ProductName, C.CategoryName; -- sort by product then category
13
14
```

100 % No issues found

Results Messages

	ProductName	CategoryName	Amount	PurchaseDate
1	Apple	Food	10.00	2026-01-03
2	Apple	Food	10.00	2026-01-03
3	Basketball	Health and Beauty Aids	250.00	2026-01-11
4	Body Lotion	Electronics	90.00	2026-01-11
5	Body Lotion	Electronics	90.00	2026-01-14
6	Body Lotion	Electronics	90.00	2026-01-20
7	Car Oil	Sports	35.00	2026-01-19
8	Car Oil	Sports	35.00	2026-01-23
9	Chicken Breast	Food	18.00	2026-01-06
10	Chicken Breast	Food	18.00	2026-01-11
11	Conditioner	Electronics	25.00	2026-01-07
12	Conditioner	Electronics	25.00	2026-01-07
13	Conditioner	Electronics	25.00	2026-01-13
14	Headphones	Electronics	2.00	2026-01-05
15	Headphones	Electronics	2.00	2026-01-09

4) Display all the products in each category. [1]

SQLQuery6.s...critt (59))* ✕

```
1  -- 4) Display all the products in each category.
2
3  -- List all products grouped by their category
4  SELECT
5      C.CategoryName,
6      P.ProductName
7  FROM Products P
8  JOIN Categories C ON P.CategoryID = C.CategoryID;
9
10
```

100 % No issues found

Results Messages

	CategoryName	ProductName
1	Electronics	Laptop
2	Electronics	Phone
3	Food	Apple
4	Electronics	Laptop
5	Electronics	Phone
6	Electronics	Tablet
7	Electronics	Headphones
8	Electronics	Smartwatch
9	Food	Apple
10	Food	Bread
11	Food	Milk
12	Food	Chicken Breast
13	Food	Rice
14	Electronics	Shampoo
15	Electronics	Conditioner

5) Display the product name and category sorted by category then by product. [1]

SQLQuery6.s...critt (59))* ✕

```
1  -- 5) Display the product name and category sorted by category then by product.
2
3  -- Display product and category, sorted by category first, then product name
4  SELECT
5      P.ProductName,
6      C.CategoryName
7  FROM Products P
8  JOIN Categories C ON P.CategoryID = C.CategoryID
9  ORDER BY C.CategoryName, P.ProductName;
10
11
```

100 % No issues found

Results Messages

	ProductName	CategoryName
1	Blender	Clothing
2	Microwave	Clothing
3	Vacuum Cleaner	Clothing
4	Body Lotion	Electronics
5	Conditioner	Electronics
6	Headphones	Electronics
7	Laptop	Electronics
8	Laptop	Electronics
9	Phone	Electronics
10	Phone	Electronics
11	Shampoo	Electronics
12	Smartwatch	Electronics
13	Tablet	Electronics
14	Toothpaste	Electronics
15	Apple	Food
16	Apple	Food
17	Bread	Food
18	Chicken Breast	Food
19	Jacket	Food
20	Jeans	Food
21	Milk	Food

6) Display a count of the number of customers in each city sorted by city. [1]

SQLQuery6.s...critt (59))* ✕

```
1  -- 6) Display a count of the number of customers in each city sorted by city.
2
3  -- Count customers per city and sort alphabetically by city
4  SELECT
5      City,
6      COUNT(*) AS TotalCustomers
7  FROM Customers
8  GROUP BY City
9  ORDER BY City;
10
11
```

100 % No issues found

Results Messages

	City	TotalCustomers
1	Boston	5
2	Chicago	6
3	Dallas	2
4	Los Angeles	2
5	Miami	3
6	New York	4
7	Seattle	2

7) Insert a new category "Health and Beauty Aids" with an appropriate description into the category table. (show me the sql and result set). [1]

SQLQuery6.s...critt (59))* ✕

```

1  -- 7) Insert a new category "Health and Beauty Aids" with an appropriat
2
3  INSERT INTO Categories (CategoryName, Description)
4  VALUES ('Health and Beauty Aids', 'Personal care products');
5  SELECT * FROM Categories;
6
7

```

100 % No issues found

Results Messages

	CategoryID	CategoryName	Description
1	1	Electronics	Devices
2	2	Food	Groceries
3	3	Electronics	Devices and gadgets
4	4	Food	Groceries and consumables
5	5	Health and Beauty Aids	Personal care products
6	6	Clothing	Apparel and fashion
7	7	Sports	Athletic equipment
8	8	Home Goods	Household items
9	9	Automotive	Car-related products
10	10	Books	Printed and digital books
11	11	Electronics	Devices and gadgets
12	12	Food	Groceries and consumables
13	13	Health and Beauty Aids	Personal care products
14	14	Clothing	Apparel and fashion
15	15	Sports	Athletic equipment
16	16	Home Goods	Household items
17	17	Automotive	Car-related products
18	18	Books	Printed and digital books
19	19	Electronics	Devices and gadgets
20	20	Food	Groceries and consumables
21	21	Health and Beauty Aids	Personal care products

8) Insert records that indicates the purchases Judy Denth and Idris Elba made from Health and Beauty Aids and produce a report. Include the SQL to perform the inserts as well. [1]

SQLQuery6.s...critt (59))* ✕

```

1  -- 8) Insert records that indicates the purchases Judy Denth and Idris Elba made from Health and Beauty Aids
2  -- and produce a report. Include the SQL to perform the inserts as well. [1]
3
4  -- Get the ID of the new category
5  DECLARE @CatID INT;
6  SELECT @CatID = CategoryID
7  FROM Categories
8  WHERE CategoryName = 'Health and Beauty Aids';
9
10 -- Insert a product into that category
11 INSERT INTO Products (ProductName, CategoryID)
12 VALUES ('Shampoo', @CatID);
13
14 -- Get the ProductID we just inserted
15 DECLARE @ProdID INT;
16 SELECT @ProdID = ProductID
17 FROM Products
18 WHERE ProductName = 'Shampoo' AND CategoryID = @CatID;
19
20 -- Insert purchases for two customers
21 INSERT INTO Purchases (CustomerID, ProductID, PurchaseDate, Amount)
22 VALUES
23 (1, @ProdID, GETDATE(), 15),
24 (2, @ProdID, GETDATE(), 20);
25
26 -- Generate report for that category
27 SELECT
28     C.FirstName,
29     C.LastName,
30     P.ProductName,
31     PU.Amount
32 FROM Purchases PU
33 JOIN Customers C ON PU.CustomerID = C.CustomerID
34 JOIN Products P ON PU.ProductID = P.ProductID
35 JOIN Categories CA ON P.CategoryID = CA.CategoryID
36 WHERE CA.CategoryName = 'Health and Beauty Aids';
37

```

70 % No issues found

Results Messages

	FirstName	LastName	ProductName	Amount
1	Scarlett	Johansson	Basketball	250.00
2	Judy	Denth	Shampoo	15.00
3	Idris	Elba	Shampoo	20.00

9) Display the category, purchase date and amount for each recipient include LName, FName) for each category sorted by Lname that have purchased at least 4 products. [1]

SQLQuery6.s...critt (59))* ✕

```

1  -- 9) Display the category, purchase date and amou
2
3  -- Show customers who made at least 4 purchases, i
4  SELECT
5      C.LastName,
6      C.FirstName,
7      CA.CategoryName,
8      PU.PurchaseDate,
9      PU.Amount
10 FROM Purchases PU
11 JOIN Customers C ON PU.CustomerID = C.CustomerID
12 JOIN Products P ON PU.ProductID = P.ProductID
13 JOIN Categories CA ON P.CategoryID = CA.CategoryID
14 WHERE C.CustomerID IN (
15     -- filter customers with 4 or more purchases
16     SELECT CustomerID
17     FROM Purchases
18     GROUP BY CustomerID
19     HAVING COUNT(*) >= 4
20 )
21 ORDER BY C.LastName;
22
23

```

100 % No issues found

	LastName	FirstName	CategoryName	PurchaseDate	Amount
1	Denth	Judy	Electronics	2026-01-01	1200.00
2	Denth	Judy	Electronics	2026-01-02	800.00
3	Denth	Judy	Electronics	2026-01-05	200.00
4	Denth	Judy	Food	2026-01-06	15.00
5	Denth	Judy	Electronics	2026-01-07	25.00
6	Denth	Judy	Health and Beauty Aids	2026-04-30	15.00
7	Elha	Idris	Food	2026-01-03	10.00

10) Display a count of the number of products for each category using one SQL statement. [1]

SQLQuery6.s...critt (59))* ✕

```

1  -- 10) Display a count of the number of products for each category using
2
3  -- Count number of products in each category (include empty categories)
4  SELECT
5      C.CategoryName,
6      COUNT(P.ProductID) AS TotalProducts
7  FROM Categories C
8  LEFT JOIN Products P ON C.CategoryID = P.CategoryID
9  GROUP BY C.CategoryName;

```

100 % No issues found

	CategoryName	TotalProducts
1	Automotive	0
2	Books	0
3	Clothing	3
4	Electronics	11
5	Food	10
6	Health and Beauty Aids	4
7	Home Goods	2
8	Sports	2

Administrative tasks

11) Display the active logins in SQL Server. [1]

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a query in the SQL Query window:

```
1  -- Show active SQL Server sessions (current logins)
2  SELECT
3      session_id,
4      login_name,
5      status,
6      host_name,
7      program_name
8  FROM sys.dm_exec_sessions
9  WHERE is_user_process = 1;
```

The bottom pane shows the results of the query, displaying a table with 9 rows and 6 columns: login_name, status, host_name, session_id, and program_name.

	login_name	status	host_name	session_id	program_name
1	Ethanslaptop\critt	sleeping	ETHANSLAPTOP	54	SQL Server Management Studio
2	Ethanslaptop\critt	sleeping	ETHANSLAPTOP	55	SQL Server Management Studio
3	Ethanslaptop\critt	running	ETHANSLAPTOP	59	SQL Server Management Studio
4	Ethanslaptop\critt	sleeping	ETHANSLAPTOP	74	Microsoft SQL Server Management Studio - Transa...
5	Ethanslaptop\critt	sleeping	ETHANSLAPTOP	87	SQL Server Management Studio
6	Ethanslaptop\critt	sleeping	ETHANSLAPTOP	90	SQL Server Management Studio
7	Ethanslaptop\critt	sleeping	ETHANSLAPTOP	97	SQL Server Management Studio
8	Ethanslaptop\critt	sleeping	ETHANSLAPTOP	103	SQL Server Management Studio
9	Ethanslaptop\critt	sleeping	ETHANSLAPTOP	104	SQL Server Management Studio

12) Demonstrate how to terminate a connection to SQL Server. [1]

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a query in the SQL Query window:

```
1  -- 12) Demonstrate how to terminate a connection to SQL Server.
2
3  -- Terminate a session by its session ID
4  KILL 54;
5
6
```

The bottom pane shows the results of the query, displaying a table with 1 row and 1 column: login_name.

login_name
Ethanslaptop\critt

Below the table, the status bar indicates: "Commands completed successfully." and "Completion time: 2026-04-30T19:44:05.9434998-04:00".

13) Create a trigger for update, insert, delete on a table. Trigger writes message to an audit table that provides 'useful' information. [3]

```
SQLQuery6.s...critt (59))* ✕
1  -- 13) Create a trigger for update, insert, delete on a table. Trigger writes message to an audi
2
3  -- Audit table to track changes
4  CREATE TABLE AuditLog (
5      AuditID INT IDENTITY PRIMARY KEY,
6      ActionType VARCHAR(10), -- INSERT, UPDATE, DELETE
7      TableName VARCHAR(50),
8      ActionDate DATETIME DEFAULT GETDATE()
9  );
10
11  GO
12
13  -- Trigger to log changes on Purchases table
14  CREATE TRIGGER trg_Audit_Purchases
15  ON Purchases
16  AFTER INSERT, UPDATE, DELETE
17  AS
18  BEGIN
19      INSERT INTO AuditLog (ActionType, TableName)
20      SELECT
21          CASE
22              WHEN EXISTS(SELECT * FROM inserted) AND EXISTS(SELECT * FROM deleted) THEN 'UPDATE'
23              WHEN EXISTS(SELECT * FROM inserted) THEN 'INSERT'
24              ELSE 'DELETE'
25          END,
26          'Purchases';
27  END;
28
70 %  No issues found
Messages
Commands completed successfully.
Completion time: 2026-04-30T19:46:10.2705763-04:00
```

14) Create a trigger that performs a cascading delete in a second table when a record is deleted from a table.

```
SQLQuery6.s...critt (59))* ✕
1  -- 14) Create a trigger that performs a cascading delete in a second
2
3  -- Trigger to delete related purchases when a customer is deleted
4  CREATE TRIGGER trg_DeleteCustomer
5  ON Customers
6  AFTER DELETE
7  AS
8  BEGIN
9      DELETE FROM Purchases
10     WHERE CustomerID IN (SELECT CustomerID FROM deleted);
11  END;
12
13

100 %  No issues found
Messages
Commands completed successfully.
Completion time: 2026-04-30T19:46:50.1182616-04:00
```

15) SQL to backup a database. [1]

```
SQLQuery6.s...critt (59))* ✕
1  -- 15) SQL to backup a database. [1]
2
3  -- Backup database to disk file
4  BACKUP DATABASE SalesDB
5  TO DISK = 'C:\Backup\SalesDB.bak';
6
7
```

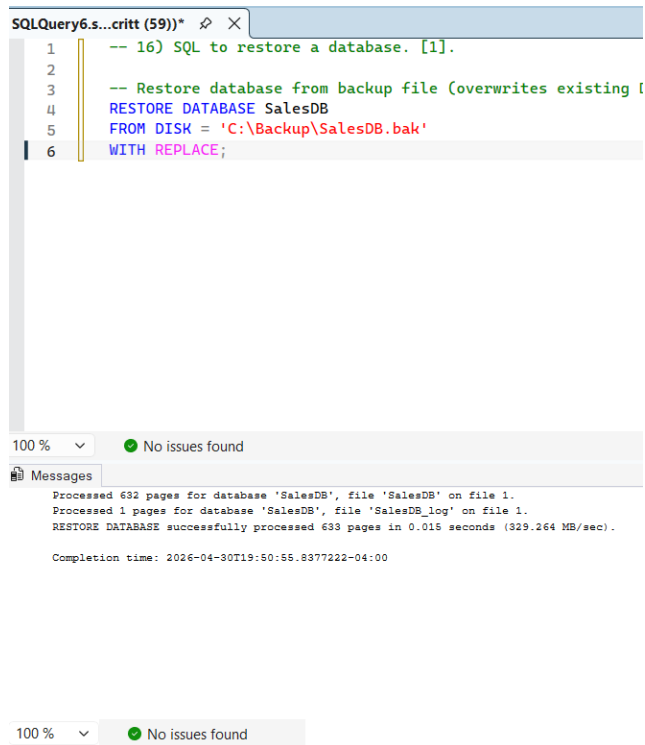
100 % ✓ No issues found

Messages

Processed 632 pages for database 'SalesDB', file 'SalesDB' on file 1.
Processed 1 pages for database 'SalesDB', file 'SalesDB_log' on file 1.
BACKUP DATABASE successfully processed 633 pages in 0.026 seconds (189.960 MB/sec).

Completion time: 2026-04-30T19:48:22.3524592-04:00

16) SQL to restore a database. [1]. Bonus point - back up a database on one server and restore on another. [1]



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query window with the following text:

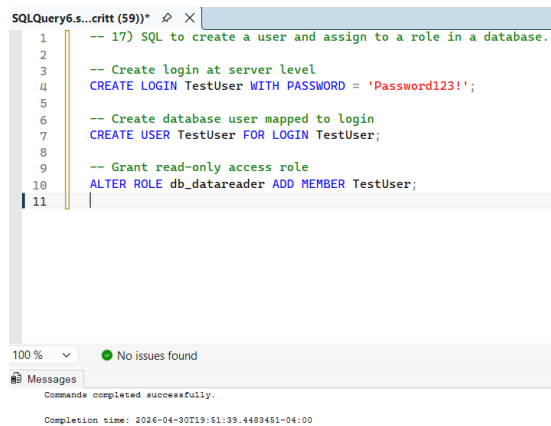
```
-- 16) SQL to restore a database. [1].  
  
-- Restore database from backup file (overwrites existing)  
RESTORE DATABASE SalesDB  
FROM DISK = 'C:\Backup\SalesDB.bak'  
WITH REPLACE;
```

The bottom pane shows the Messages tab with the following output:

```
Processed 632 pages for database 'SalesDB', file 'SalesDB' on file 1.  
Processed 1 pages for database 'SalesDB', file 'SalesDB_log' on file 1.  
RESTORE DATABASE successfully processed 633 pages in 0.015 seconds (329.264 MB/sec).  
  
Completion time: 2026-04-30T19:50:55.8377222-04:00
```

Below the screenshot, there is a second, identical-looking interface snippet, likely representing a second step or a duplicate of the first.

17) SQL to create a user and assign to a role in a database. [1]



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query window with the following text:

```
-- 17) SQL to create a user and assign to a role in a database.  
  
-- Create login at server level  
CREATE LOGIN TestUser WITH PASSWORD = 'Password123!';  
  
-- Create database user mapped to login  
CREATE USER TestUser FOR LOGIN TestUser;  
  
-- Grant read-only access role  
ALTER ROLE db_datareader ADD MEMBER TestUser;
```

The bottom pane shows the Messages tab with the following output:

```
Commands completed successfully.  
  
Completion time: 2026-04-30T19:51:39.4489451-04:00
```